

STRATEGY PATTERN

Behavioural Design Pattern

Problem Statement

Imagine you're building an app like **Flipkart** or **Amazon**.

Customers can get different types of discounts:

- **Diwali Sale** – 20% off on everything
- **Student Discount** – 15% off with college ID
- **First Order** – ₹100 flat discount

Without Strategy Pattern

Every time a new discount type comes, you have to modify this function.

Too many if-else statements make code messy.

Hard to test each discount type separately.

Violates **Open/Closed** Principle (we learned in SOLID!)

Problems in Code

Issues with the **if-else** Approach:

- **Too Many Responsibilities:** The DiscountService class is doing two jobs. Deciding which discount to apply AND calculating the discount. It should only do one job (violates Single Responsibility Principle).
- **Hard to Extend:** Every time you want to add a new discount type (like Republic Day sale, Holi offer, Monsoon sale), you have to open the DiscountService class and add more if-else statements. This means changing existing working code.
- **Messy Code:** As you keep adding more discount types, the if-else chain becomes longer and longer, making the code harder to read, understand, and maintain.